ServiceNow MCP Bridge and MCPorter Runbook

Version: 1.0
Last Updated: 2026-11-06
Audience: Engineers onboarding to the ServiceNow MCP bridge project

1. Executive Summary

This runbook explains why MCPorter was required in this project, how it was introduced, and h

The target user experience is:

1. User writes a natural-language prompt.
2. System routes to the correct ServiceNow MCP tool.
3. System returns formatted results only.
4. User does not run low-level tool commands.

2. Why MCPorter Was Needed

2.1 Original objective

The project needed a local MCP bridge that:

1. Runs as an MCP server over stdio.
2. Connects to ServiceNow MCP over HTTP.
3. Authenticates with OAuth client credentials.
4. Proxies ‘initialize‘, ‘tools/list‘, and ‘tools/call‘.

2.2 Constraint encountered

Direct VS Code MCP usage was blocked by enterprise policy in this environment. Even with loca

2.3 Why MCPorter solved it

MCPorter provided a terminal-accessible MCP client path that allowed:

1. Listing MCP tools.
2. Calling tools deterministically.
3. Testing connectivity/auth/proxy independent of editor policy.
4. Fast diagnosis of payload/timeout/response-shape issues.

In short, MCPorter became the practical test and operations harness while preserving the same

3. Solution Architecture

3.1 Components

1. Local Node.js bridge (‘dist/index.js‘) exposing stdio MCP server.
2. ServiceNow MCP HTTP endpoint.
3. OAuth token manager with refresh/retry.
4. Prompt router and orchestrator for natural-language requests.
5. Formatted result rendering for CLI output.

3.2 High-level flow

1. User runs ‘npm run prompt -- "..."‘.
2. Prompt router chooses tool and parameters.
3. Orchestrator calls ServiceNow MCP through bridge runtime.
4. Response is normalized and formatted.
5. User sees only final result.

4. Repository Structure

Key files and folders:

1. ‘src/index.ts‘ - stdio MCP bridge server.
2. ‘src/config.ts‘ - env parsing and validation.
3. ‘src/oauthTokenManager.ts‘ - OAuth token lifecycle.
4. ‘src/mcpHttpClient.ts‘ - remote MCP HTTP client and parsing.
5. ‘src/proxy.ts‘ - proxy setup from env.
6. ‘src/promptRouter.ts‘ - prompt entrypoint and output formatting.
7. ‘src/agent/router/intentRouter.ts‘ - intent-to-tool mapping.
8. ‘src/agent/orchestrator/promptRunner.ts‘ - execution and normalization.
9. ‘src/agent/orchestrator/mcpRuntimeClient.ts‘ - in-process tool runtime client.
10. ‘.mcp.json‘ - MCPorter server configuration.
11. ‘.env.example‘ - required and optional environment variables.
12. ‘README.md‘ - project usage reference.

5. Prerequisites

1. Windows PowerShell (or compatible shell).
2. Node.js 20+.
3. npm 10+.
4. ServiceNow OAuth client credentials.
5. ServiceNow MCP endpoint URL.
6. Corporate proxy details if required.

6. One-time Project Setup

6.1 Install dependencies

‘‘‘powershell
npm install
‘‘‘

6.2 Create ‘.env‘

Create ‘.env‘ in repo root with real values.

Template:

‘‘‘dotenv
SN_INSTANCE_URL=https://your-instance.service-now.com
SN_MCP_URL=https://your-instance.service-now.com/sncapps/mcp-server/mcp/sn_mcp_server_default
SN_OAUTH_TOKEN_URL=https://your-instance.service-now.com/oauth_token.do
SN_CLIENT_ID=your_client_id
SN_CLIENT_SECRET=your_client_secret
SN_AUTH_MODE=oauth

```
SN_OAUTH_SCOPE=

LOG_LEVEL=info

HTTPS_PROXY=http://localhost:9000
HTTP_PROXY=http://localhost:9000
NO_PROXY=localhost,127.0.0.1

MCPORTER_CALL_TIMEOUT=180000
PROMPT_VERBOSE=0
PROMPT_CACHE_LOOKUP_TTL_MS=60000
PROMPT_CACHE_SUMMARY_TTL_MS=600000
PROMPT_SUMMARY_SOFT_TIMEOUT_MS=30000
```

Important:

1. Never commit `.env`.
2. Rotate secrets if they were exposed in logs/chat.

6.3 Build project

```powershell
npm run build
```

7. Running and Testing Paths

7.1 Preferred end-user path (prompt only)

Use this for production-like UX testing:

```powershell
npm run prompt -- "show latest 5 incidents"
npm run prompt -- "please summarize INC1639945"
npm run prompt -- "show latest 5 incidents assigned to A Alexson Hentry"
```

Expected behavior:

1. Formatted output only.
2. No raw envelope unless verbose mode is enabled.

7.2 Bridge runtime path

```powershell
npm run dev
```

or

```powershell
npm run build
npm run start
```

‘‘‘

7.3 Smoke test path

‘‘‘powershell
npm run test:smoke
‘‘‘

Validates:

1. OAuth token retrieval.
2. ‘initialize‘.
3. ‘tools/list‘.

7.4 MCPorter direct path (diagnostics only)

Use this when debugging protocol/tool issues, not for standard user flow.

List tools:

‘‘‘powershell
npx --yes mcporter --config ".\.mcp.json" list servicenow-local-bridge --schema --timeout 120
‘‘‘

Call lookup:

‘‘‘powershell
npx --yes mcporter --config ".\.mcp.json" call servicenow-local-bridge.look_up_incident_recor
‘‘‘

Call summarize:

‘‘‘powershell
npx --yes mcporter --config ".\.mcp.json" call servicenow-local-bridge.incident_summarization
‘‘‘

8. Why Long Env Prefix Commands Appeared Earlier

During troubleshooting, commands were run with inline ‘$env:‘ assignments for reliability acr

Current best practice:

1. Put all values in ‘.env‘.
2. Prompt command auto-loads ‘.env‘.
3. Run only short prompt commands.

9. Prompt Behavior and Dynamic Logic

Supported intent behaviors:

1. Incident number detected (‘INC...‘) -> summarize tool.
2. Non-incident prompt -> lookup tool.
3. Dynamic extraction of limit and common filters (status/priority).
4. Dynamic extraction of ‘assigned_to‘ names from prompt text.

Note on boundaries:

1. Filtering by assignment group requires backend tool support for ‘assignment_group‘.
2. Current tool schema reliably supports ‘assigned_to‘ user matching.

10. Output Modes

Default mode:

1. Results only (formatted table or summary text).
2. No internal metrics envelope printed.

Verbose mode:

‘‘‘powershell
$env:PROMPT_VERBOSE="1"
npm run prompt -- "show latest 5 incidents"
‘‘‘

Verbose includes internal envelope and metrics.

11. Performance and Reliability Features

1. OAuth token caching and refresh buffer.
2. Retry once on HTTP 401.
3. MCP response parsing for JSON/SSE/wrapped variants.
4. Prompt response cache with disk persistence:
   1. file: ‘.cache/prompt-cache.json‘
   2. lookup TTL default: 60s
   3. summary TTL default: 10m
5. Slow summary warning threshold.
6. Summarization fallback to lookup when possible.

12. Proxy Guidance

If your network needs a proxy:

1. Set ‘HTTPS_PROXY‘, optionally ‘HTTP_PROXY‘.
2. Keep ‘NO_PROXY=localhost,127.0.0.1‘.
3. Verify proxy listener:

‘‘‘powershell
Test-NetConnection localhost -Port 9000
‘‘‘

13. Troubleshooting Guide

13.1 No results returned

1. Validate user/group value exists in ServiceNow.
2. Check whether prompt asked for unsupported field (for example ‘assignment_group‘).
3. Run direct lookup with a known assignee.

13.2 Prompt too slow

1. Summarization can be naturally slow upstream.
2. Increase ‘MCPORTER_CALL_TIMEOUT‘.
3. Re-run same prompt to use cache.
4. Keep summary timeout high for production realism.

13.3 Unexpected raw JSON

1. Ensure default prompt mode is used.
2. Ensure ‘PROMPT_VERBOSE‘ is not ‘1‘.
3. Pull latest code where table formatter is included.

13.4 Auth failures

1. Verify client ID/secret/token URL.
2. Confirm OAuth client grants are enabled.
3. Rotate secret if exposure occurred.

13.5 VS Code asks for Allow during agent-executed commands

1. This is VS Code agent security gating.
2. It is not application behavior.
3. End users in production should interact with prompt interface, not run agent-executed term

14. Security Checklist

1. ‘.env‘ remains git-ignored.
2. No secrets in code or docs.
3. Secret rotation policy defined.
4. Redacted logs enabled.
5. Least-privilege ServiceNow integration user.

15. Operational SOP for New Team Members

Day-1 onboarding:

1. Clone repo.
2. ‘npm install‘.
3. Copy ‘.env.example‘ to ‘.env‘ and populate real values.
4. ‘npm run build‘.
5. ‘npm run test:smoke‘.
6. ‘npm run prompt -- "show latest 5 incidents"‘.
7. ‘npm run prompt -- "please summarize INC1639945"‘.

Daily usage:

1. Use prompt commands only.
2. Avoid direct MCPorter commands unless debugging.
3. Keep outputs in results-only mode.

16. Commands Reference (Quick Copy)

Build:

‘‘‘powershell
npm run build
‘‘‘

Smoke test:

‘‘‘powershell
npm run test:smoke
‘‘‘

Latest incidents:

‘‘‘powershell
npm run prompt -- "show latest 10 incidents"
‘‘‘

Assignee filter:

‘‘‘powershell
npm run prompt -- "show latest 5 incidents assigned to A Alexson Hentry"
‘‘‘

Summary:

‘‘‘powershell
npm run prompt -- "please summarize INC1639945"
‘‘‘

Verbose mode:

‘‘‘powershell
$env:PROMPT_VERBOSE="1"
npm run prompt -- "show latest 5 incidents"
‘‘‘

17. Change History Highlights

1. Bridge and proxy/auth stack implemented.
2. MCP response compatibility hardening added.
3. Prompt router/orchestrator introduced.
4. Results-only output mode enabled.
5. Table formatting for lookup results enabled.
6. ‘.env‘ auto-load for prompt mode enabled.
7. In-process runtime execution path added for production-like prompt testing.

18. Handover Notes

For production rollout alignment:

1. Prefer always-on service deployment for bridge.
2. Integrate secrets from secure secret store.
3. Keep prompt UX stable and command-free for end users.
4. Extend tool schema in ServiceNow for richer fields like ‘assignment_group‘ when required.